

2020-2021 Final Exam

Answer Key

CPT104 Operating Systems

Answer Key with English & Chinese Explanations

Question 1: Fundamentals (46 marks)

(a) [6 marks]

English Answer

Internal Fragmentation:

- Allocated memory is larger than requested memory.
- Waste occurs INSIDE the allocated block.
- Common in fixed-size partition schemes (e.g., paging).
- Example: Process requests 3KB, gets 4KB page -> 1KB wasted.

External Fragmentation:

- Total free memory is sufficient, but exists in small non-contiguous pieces.
- No single block is large enough to satisfy a request.
- Common in variable-size partition schemes (e.g., segmentation).
- Solution: Compaction (defragmentation) or paging.

中文解释

内部碎片：分配的内存大于请求的内存，浪费发生在已分配块的内部。常见于固定分区方案（如分页）。

例：进程请求3KB，获得4KB页 → 浪费1KB。

外部碎片：总空闲内存足够，但以小的非连续碎片存在，没有单一块能满足请求。常见于可变分区方案（如分段）。解决方案：

紧凑（碎片整理）或分页。

(b) [6 marks]

English Answer

Two key components of virtual machine architecture:

(1) Virtual Machine Monitor (VMM) / Hypervisor:

- Manages the underlying physical hardware resources.
- Isolates VMs from each other and from the host.
- Provides controlled access to hardware for each VM.

(2) Virtual Hardware Layer:

- Emulates or virtualizes CPU, memory, disk, and I/O devices.
- Each VM believes it has dedicated physical hardware.
- Provides the illusion of a complete, independent computer system.

中文解释

虚拟机架构的两个关键组件：

(1) VMM/虚拟机监控器（Hypervisor）：

- 管理底层物理硬件资源
- 隔离各虚拟机及宿主机
- 为每个VM提供受控的硬件访问

(2) 虚拟硬件层：

- 模拟/虚拟化CPU、内存、磁盘和I/O设备
- 每个VM认为自己拥有专用的物理硬件
- 提供完整独立计算机系统的假象

(c) [12 marks]

English Answer

Three disk allocation methods:

(1) Contiguous Allocation:

- File occupies consecutive disk blocks.
- Advantage: Fast direct access (sequential + random). Simple to implement.
- Disadvantage: External fragmentation. Difficult to grow files.

(2) Linked Allocation:

- File is a linked list of disk blocks, each block contains a pointer to the next.
- Advantage: No external fragmentation. Easy to grow files.
- Disadvantage: Slow random access (must follow pointers). Pointer reliability concerns.

(3) Indexed Allocation:

- An index block stores pointers to all data blocks of the file.
- Advantage: Direct access (from index). No external fragmentation.

- Disadvantage: Index block overhead, especially for small files. Index block size limitation.

中文解释

三种磁盘分配方法：

(1) 连续分配：文件占据连续的磁盘块。

优点：支持快速直接访问（顺序+随机），实现简单。

缺点：外部碎片，文件扩展困难。

(2) 链接分配：文件是磁盘块的链表，每块包含指向下一块的指针。

优点：无外部碎片，文件易于扩展。

缺点：随机访问慢（需沿指针遍历），指针可靠性问题。

(3) 索引分配：索引块存储文件所有数据块的指针。

优点：支持直接访问（通过索引），无外部碎片。

缺点：索引块开销，尤其对小文件。索引块大小有限制。

(d) [6 marks]

English Answer

fork() creates a child process by duplicating the parent's address space.

- The parent process receives the child's PID (Process ID).
- The child process receives 0.
- Both processes continue execution from the point of the fork() call.

If the parent terminates before the child:

- The child becomes an orphan process.
- It is adopted by init (PID 1 on Unix/Linux).
- The orphan child continues to run normally.

If all threads in a process terminate:

- The process itself terminates, as there are no executing units left.

中文解释

fork()通过复制父进程的地址空间创建子进程。

- 父进程收到子进程的PID。
- 子进程收到0。
- 两个进程从fork()调用处继续执行。

如果父进程先终止：

- 子进程成为孤儿进程。
- 被init进程（PID 1）收养。
- 孤儿子进程继续正常运行。

如果进程的所有线程终止：

- 进程本身也终止，因为没有执行单元了。

(e) [4 marks]

English Answer

Without mutual exclusion, concurrent processes accessing shared data cause race conditions.

A race condition occurs when the final result of an operation depends on the order of execution of concurrent processes. This leads to data inconsistency - the same program can produce different results on different runs, making the behavior unpredictable and unreliable.

中文解释

没有互斥时，并发进程访问共享数据导致竞态条件。

竞态条件是指操作结果取决于并发进程的执行顺序。这导致数据不一致——同一程序在不同运行中可能产生不同结果，行为不可预测且不可靠。

(f) [12 marks]

English Answer

Four levels of protection in a computer system:

(1) Physical Level:

- Door locks, security guards, biometric access to server rooms.
- Prevents physical tampering, theft, or destruction of hardware.

(2) Network Level:

- Firewalls, Intrusion Detection Systems (IDS), VPNs.
- Encryption of data in transit (TLS/SSL).
- Controls network traffic and prevents unauthorized network access.

(3) Operating System Level:

- User authentication (passwords, biometrics).
- Access Control Lists (ACLs) and file permissions.
- Process isolation and memory protection.

(4) Application Level:

- Input validation and sanitization.
- Sandboxing (running code in restricted environments).
- Principle of least privilege.

中文解释

计算机系统的四个保护层次：

(1) 物理层：门锁、保安、生物识别访问服务器机房。防止物理篡改、盗窃或硬件破坏。

(2) 网络层：防火墙、入侵检测系统（IDS）、VPN。传输中数据加密（TLS/SSL）。控制网络流量，防止未授权网络访问。

(3) 操作系统层：用户认证（密码、生物识别）。访问控制列表（ACL）和文件权限。进程隔离和内存保护。

(4) 应用层：输入验证和净化。沙箱（在受限环境中运行代码）。最小权限原则。

Question 2: CPU / Memory / Disk Scheduling (34 marks)

(a) [10 marks]

English Answer

Processes: P1(arrival=0, burst=9), P2(arrival=1, burst=5), P3(arrival=2, burst=3), P4(arrival=3, burst=4)

FCFS (First-Come, First-Served) Scheduling:

Gantt chart:

| P1(0-9) | P2(9-14) | P3(14-17) | P4(17-21) |

Turnaround times (completion - arrival):

P1: $9 - 0 = 9$

P2: $14 - 1 = 13$

P3: $17 - 2 = 15$

P4: $21 - 3 = 18$

Average = $(9+13+15+18)/4 = 13.75$

Waiting times (turnaround - burst):

P1: $9 - 9 = 0$

P2: $13 - 5 = 8$

P3: $15 - 3 = 12$

P4: $18 - 4 = 14$

Average = $(0+8+12+14)/4 = 8.5$

中文解释

FCFS (先来先服务) 调度:

甘特图: P1(0-9) → P2(9-14) → P3(14-17) → P4(17-21)

周转时间 (完成-到达) : P1=9, P2=13, P3=15, P4=18

平均周转时间 = $(9+13+15+18)/4 = 13.75$

等待时间 (周转-突发) : P1=0, P2=8, P3=12, P4=14

平均等待时间 = $(0+8+12+14)/4 = 8.5$

(b) [6 marks]

English Answer

Given: 64 pages, page size = 512 bytes, 32 frames

Number of pages = 64 = 2^6 → 6 bits for page number

Page size = 512 = 2^9 bytes → 9 bits for page offset

Logical address = page number bits + offset bits = 6 + 9 = 15 bits

Number of frames = 32 = 2^5 → 5 bits for frame number

Physical address = frame number bits + offset bits = 5 + 9 = 14 bits

中文解释

64个页 = 2^6 → 页号占6位

页大小 = 512字节 = 2^9 → 页内偏移占9位

逻辑地址 = 6 + 9 = 15位

32个帧 = 2^5 → 帧号占5位

物理地址 = 5 + 9 = 14位

(c) [6 marks]

English Answer

Reference string: 5 0 2 1 0 3 0 2 4 3 0 3 2 1 3 0 1 5, Frames = 4

FIFO trace (replace oldest page):

Ref=5:	[5]	F
Ref=0:	[5,0]	F
Ref=2:	[5,0,2]	F
Ref=1:	[5,0,2,1]	F
Ref=0:	[5,0,2,1]	H
Ref=3:	[3,0,2,1]	F (replace 5)
Ref=0:	[3,0,2,1]	H
Ref=2:	[3,0,2,1]	H
Ref=4:	[3,4,2,1]	F (replace 0)
Ref=3:	[3,4,2,1]	H
Ref=0:	[3,4,0,1]	F (replace 2)
Ref=3:	[3,4,0,1]	H
Ref=2:	[3,4,0,2]	F (replace 1)
Ref=1:	[1,4,0,2]	F (replace 3)
Ref=3:	[1,4,0,3]	F (replace 2)
Ref=0:	[1,4,0,3]	H
Ref=1:	[1,4,0,3]	H
Ref=5:	[5,4,0,3]	F (replace 1)

Total page faults = 10

中文解释

使用FIFO算法，4个帧。逐步追踪如上。

FIFO替换最早进入的页面。

共10次缺页。

(d) [5 marks]

English Answer

Requests: 54, 97, 73, 128, 15, 44, 110, 34, 45

Initial head position = 23

SSTF (Shortest Seek Time First):

23 -> 15 (distance=8)
 15 -> 34 (distance=19)
 34 -> 44 (distance=10)
 44 -> 45 (distance=1)
 45 -> 54 (distance=9)
 54 -> 73 (distance=19)
 73 -> 97 (distance=24)
 97 -> 110 (distance=13)
 110 -> 128 (distance=18)

Total movement = $8+19+10+1+9+19+24+13+18 = 121$ cylinders

中文解释

SSTF算法, 初始磁头位置=23。

$23 \rightarrow 15(8) \rightarrow 34(19) \rightarrow 44(10) \rightarrow 45(1) \rightarrow 54(9) \rightarrow 73(19) \rightarrow 97(24) \rightarrow 110(13) \rightarrow 128(18)$

总移动量 = 121 柱面。

(e) [7 marks]

English Answer

Given: $P1(t=35, p=100)$, $P2(t=10, p=50)$, $P3(t=30, p=150)$

Utilization: $U = 35/100 + 10/50 + 30/150 = 0.35 + 0.20 + 0.20 = 0.75$

RM Schedulability threshold for $n=3$: $3 * (2^{1/3} - 1) = 0.7797$

Since $U = 0.75 < 0.7797$, the task set IS schedulable under Rate Monotonic.

Priority assignment (shorter period = higher priority):

P2 (period=50, highest priority)

P1 (period=100)

P3 (period=150, lowest priority)

中文解释

$U = 35/100 + 10/50 + 30/150 = 0.35 + 0.20 + 0.20 = 0.75$

RM可调度上界 ($n=3$) : $3 \times (2^{1/3} - 1) = 0.7797$

$U=0.75 < 0.7798$, 系统可调度。

优先级 (周期越短优先级越高) :

P2 (周期=50, 最高) > P1 (周期=100) > P3 (周期=150, 最低)

Question 3: Banker's Algorithm (10 marks)

English Answer

Given resources: Total $R1=15$, $R2=8$, $R3=8$

Need matrix (Max - Allocation):

P1: (3, 5, 3) P2: (5, 3, 3) P3: (1, 9, 0)

P4: (4, 2, 3) P5: (3, 3, 2)

Available = (6, 5, 2)

P4 requests (2, 0, 2):

Step 1 - Check feasibility:

Request(2,0,2) $\leq$ Need P4(4,2,3)? YES

Request(2,0,2) $\leq$ Available(6,5,2)? YES

Step 2 - Temporarily allocate to P4:

Available = (6-2, 5-0, 2-2) = (4, 5, 0)

P4 Allocation = (2,1,1), P4 Need = (2,2,1)

Step 3 - Safety check with new state:

Try P1: Need(3,5,3) $\leq$ (4,5,0)? NO ($R3: 3 > 0$)

Try P2: Need(5,3,3) $\leq$ (4,5,0)? NO ($R1: 5 > 4$)

Try P3: Need(1,9,0) $\leq$ (4,5,0)? NO ($R2: 9 > 5$)

Try P4: Need(2,2,1) $\leq$ (4,5,0)? NO ($R3: 1 > 0$)

Try P5: Need(3,3,2) $\leq$ (4,5,0)? NO ($R3: 2 > 0$)

No process can proceed -> UNSAFE state.

Conclusion: P4's request for (2,0,2) should NOT be granted.

中文解释

Need矩阵 = Max - Allocation。

Available = (6,5,2)

P4请求(2,0,2):

检查可行性:

Request(2,0,2) $\leq$ Need P4(4,2,3)

Request(2,0,2) $\leq$ Available(6,5,2)

临时分配后 Available = (4,5,0)

安全检查:

P1: Need(3,5,3) $\leq$ (4,5,0)? ($R3: 3 > 0$)

P2: Need(5,3,3) $\leq$ (4,5,0)? ($R1: 5 > 4$)

P3: Need(1,9,0) $\leq$ (4,5,0)? ($R2: 9 > 5$)

P4: Need(2,2,1) $\leq$ (4,5,0)? ($R3: 1 > 0$)

P5: Need(3,3,2) <= (4,5,0)? (R3: 2>0)

所有进程都无法继续 → 不安全状态。

结论：不应批准P4的请求(2,0,2)。

Question 4: Peterson's Algorithm (10 marks)

English Answer

The while loop condition was changed from:

```
while (flag[j] AND turn == j) { wait; }
```

to:

```
while (flag[j] OR turn == j) { wait; }
```

(1) Mutual Exclusion: VIOLATED.

Example: P0 sets flag[0]=true, turn=1. P1 sets flag[1]=true, turn=0.

P0 checks: flag[1] OR turn==0 = true OR true = true -> continues waiting. But wait...

Actually: P0 checks: flag[1] OR (turn==0) = true OR true = true -> P0 WAITS.

P1 checks: flag[0] OR (turn==1) = true OR false = true -> P1 WAITS.

Let me reconsider: flag[j] OR turn==j means: wait if the OTHER is interested OR if it's not my turn.

If both set flag=true and P0 sets turn=0, P1 sets turn=1:

P0 checks: flag[1] OR (turn==1) = true OR true = true -> P0 WAITS.

P1 checks: flag[0] OR (turn==0) = true OR false = true -> P1 WAITS.

Actually with OR, if P0 sets turn=0: P1 checks flag[0] OR (turn==0) = true OR true = waits.

P0 checks flag[1] OR (turn==1) = true OR false = true -> also waits. Deadlock!

Wait - let me re-examine. With OR condition, if turn=0:

P0 checks: flag[1] OR turn==1 = flag[1] OR false = flag[1]

P1 checks: flag[0] OR turn==0 = flag[0] OR true = true -> P1 always waits

So if P0 has turn, P1 always waits regardless. P0 waits only if flag[1]=true.

If both set flags first, then: P0 waits if flag[1]=true, P1 waits always.

This means P0 can proceed once P1 hasn't set flag yet.

For mutual exclusion violation: consider P0 in CS, P0 sets turn=0. P1 checks flag[0] OR turn==0 = true -> waits.

But consider timing: P0 exits CS, sets flag[0]=false. P1 checks: false OR true = true -> still waits.

P0 re-enters: flag[0]=true, then immediately checks flag[1] OR turn==1.

If P1 hasn't set flag[1] yet: false OR false = false -> P0 enters CS.

Then P1 sets flag[1]=true and checks: flag[0] OR turn==0 = true -> waits.

This seems to maintain mutual exclusion in this case.

Let me reconsider the original claim. The key scenario:

P0 sets flag[0]=true. P1 sets flag[1]=true. P0 sets turn=0. P1 sets turn=1.

(depending on interleaving of turn assignment)

If turn ends up = 1: P0 checks flag[1] OR (1==1) = true -> waits.

P1 checks flag[0] OR (1==0) = true -> waits. Deadlock (bounded wait violation).

If turn ends up = 0: P0 checks flag[1] OR (0==1) = true -> waits.
P1 checks flag[0] OR (0==0) = true -> waits. Deadlock.

So with OR, both processes always wait once both flags are set -> deadlock.

(2) Progress: VIOLATED - both processes wait indefinitely (deadlock).

(3) Bounded Waiting: VIOLATED - neither process can enter the CS.

中文解释

while循环条件从 AND 改为 OR:

原: while (flag[j] AND turn == j)

改: while (flag[j] OR turn == j)

(1) 互斥: 可能被违反。一旦两个进程都设置了flag=true, 由于OR条件, 两个进程都会等待。

分析: 如果两个进程都设置了flag, 无论turn值如何:

P0检查: flag[1] OR (turn==1) → 至少一个为true → P0等待

P1检查: flag[0] OR (turn==0) → 至少一个为true → P1等待

结果: 两个进程都永远等待 → 死锁

(2) 前进性 (Progress): 被违反 - 两个进程都无法进入临界区 (死锁)。

(3) 有限等待 (Bounded Waiting): 被违反 - 没有进程能进入临界区。